

Computational Tour of Nonlinear Programming

Code walkthrough of the 23 ISE 5406 notebooks

Dr. Robert Hildebrand

Spring 2026

Grado Department of Industrial & Systems Engineering
Virginia Tech

Where We Are in the Course

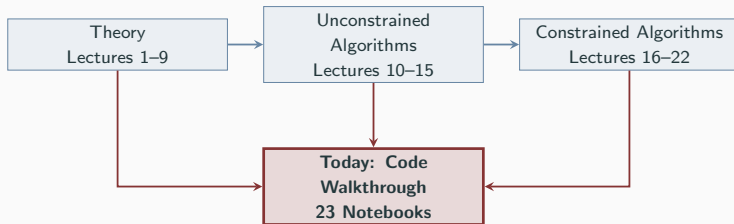
Through Lecture 22 we built the theory

- Convex analysis, KKT, duality — Lectures 1–9
- Unconstrained algorithms — line search, Newton, CG, BFGS — Lectures 10–15
- Constrained algorithms — trust region, interior point, SQP, ADMM
- Case studies — portfolio, networks, compressed sensing

Today: 23 notebooks, one code-walkthrough deck

- What every *modern* solver actually does and when to use it
- What “from scratch” implementations look like
- Empirical scaling evidence: *why* sparsity and structure matter

Big Picture: Course Roadmap



Three Parts

- Part I — Modeling languages & solver interfaces (11 notebooks)
- Part II — Implement it yourself (9 notebooks)
- Part III — Large-scale case studies (3 notebooks)

How to read these notebooks

Every notebook follows a common seven-part structure:

1. Algorithmic background — bridge to the course notes
2. Minimal working example — simplest possible use
3. Building up complexity — realistic structure
4. Under the hood — iteration diagnostics, KKT residuals, timings
5. Scaling experiment — runtime / memory vs. problem size on log-log axes
6. Check your understanding — T/F and short-answer items
7. Coding exercise — one longer open-ended task

The scaling experiment is the key. Every claim about sparsity, conditioning, and acceleration is backed by a log-log plot on real problems.

Roadmap

Part 0.5 — Python Toolchain: Packages, Environments, Conventions

Part I — Modeling Languages and Solver Interfaces

Part II — Implement It Yourself

Part III — Large-Scale Case Studies

Deep Dive — Acceleration, Momentum, and Stochastic Gradient

Summary

Before the notebooks — why this matters

The single biggest reason a course like this gets stuck

“It works on my machine, but not on yours.” Ninety percent of the time, the problem is not the math — it’s that two people have different versions of NumPy, or one has CVXPY 1.4 and the other 1.7, or a solver compiled against the wrong BLAS.

What the next few slides give you

- A vocabulary: *interpreter, package, environment, kernel*.
- A reproducible recipe for this course’s 23 notebooks.
- The specific landmines you’ll hit with `cvxpygen`, `ipopt`, `gurobi` — and how to sidestep them.

If you’ve already been doing this for years: skim and skip. If you’ve mostly `pip` installed into the system Python until now, stay with me — this part saves you pain later.

Layer 1 — The interpreter and packages

Two separate objects that beginners often conflate

- **Python interpreter** (`python3.11`, `python3.12`, ...) — the program that runs `.py` files. A given machine can have many.
- **Packages** (`numpy`, `cvxpy`, ...) — code libraries installed *for a specific interpreter*. Installing NumPy for `python3.11` does **not** install it for `python3.12`.

Two package managers you'll meet

- `pip` — the standard Python installer; reaches PyPI. Pure Python and most scientific wheels: `pip install numpy`.
- `conda` / `mamba` — manages compiled binaries too (HSL, IPOPT, BLAS, HDF5). When a package needs a C/Fortran compiler or a system library, `conda` usually has a prebuilt wheel that “just works.”

Rule of thumb. Pure-Python stuff: `pip`. Anything with C/Fortran under the hood (`ipopt`, `cyipopt`, `scipopt`): prefer `conda-forge`. Mixing `pip` and `conda` in one env works — but install `conda` stuff first.

Layer 2 — Environments: the one habit that pays off forever

An environment is a self-contained folder holding one interpreter and a specific set of packages at specific versions.

Why you want one per project

- Project A needs `cvxpy==1.4`; project B needs `cvxpy==1.7`. Without envs: impossible. With envs: two folders, no conflict.
- Accidentally `pip` installing a broken version never corrupts your other projects — it's sandboxed.
- You can delete the whole env (`rm -rf`) and rebuild from a `requirements.txt` in seconds. The env is *disposable*.

Two ways to make one (pick one)

1. Built-in venv (lightweight, no install needed)

```
python3.11 -m venv .venv
source .venv/bin/activate      # macOS/Linux
.venv\Scripts\activate        # Windows
```

2. conda (when you need IPOPT, SCIP, HDF5, ...)

```
conda create -n ise5406 python=3.11
conda activate ise5406
```


Layer 2 — Booting an environment in practice

Once an environment exists, you have to *activate* it in every new shell before `python` / `pip` / `jupyter` point at the right interpreter.

```
# Open a fresh terminal. Verify which python you'd get by default:
which python           # may be /usr/bin/python -- the system one, NOT yours

# Activate the course env:
conda activate ise5406

# Now the shell is "inside" the env:
which python           # /Users/you/miniconda3/envs/ise5406/bin/python <-- yours
pip install cvxpy      # installs into the env, not system-wide

# When you're done, step out:
conda deactivate
```

If `pip install X` ever says “...externally-managed-environment” or asks you to use `--break-system-packages`: **stop**. You're installing into the system interpreter. Activate an environment first.

VS Code / PyCharm / Cursor: tell the editor which interpreter to use (bottom-right in VS Code). It should show the env's path, not `/usr/bin/python`.

Layer 3 — Pinning versions, so it's reproducible

Activating the env is step one. You also want to *record* which packages and versions you installed, so your collaborator (or future-you) can recreate it.

The minimal workflow — all you need for this course

After installing what you need:

```
pip freeze > requirements.txt    # writes numpy==2.1.3, cvxpy==1.7.5, ...
```

On another machine / a fresh env:

```
python -m venv .venv && source .venv/bin/activate
```

```
pip install -r requirements.txt
```

A requirements.txt committed to the repo is enough for this course. If you use conda, the analogous file is environment.yml (generated via `conda env export --from-history > environment.yml`).

Jupyter, specifically — kernels are their own thing

A *Jupyter kernel* is a (name, interpreter) pair. The notebook UI shows you a list of kernels; each one launches a specific Python when you execute cells. **Installing jupyter does not automatically register your env as a kernel.**

```
# Inside the activated env, one time:
pip install ipykernel
python -m ipykernel install --user \
    --name ise5406 --display-name "Python (ISE 5406)"

# List registered kernels:
jupyter kernelspec list

# Remove one:
jupyter kernelspec uninstall ise5406
```

Symptom #1 of a missing kernel registration: you `pip install cvxpy` inside your env, open a notebook, run `import cvxpy`, and it fails with `ModuleNotFoundError`. The notebook is running some other kernel's interpreter — not your env's. Fix: register the kernel, then pick it from the kernel selector at the top of the notebook.

Conventions you'll see in every notebook

Canonical import aliases — read them as a uniform dialect

```
import numpy as np           # array math
import pandas as pd          # tabular data
import matplotlib.pyplot as plt
import scipy as sp           # or targeted: from scipy import optimize
import cvxpy as cp
```

These aren't just tradition — readers recognize `np.linalg.solve(...)` instantly; `numpy.linalg.solve(...)` feels foreign. Use them.

Two notebook-specific niceties

```
# Autoreload imported modules without restarting the kernel:
%load_ext autoreload
%autoreload 2

# Inline / interactive plots:
%matplotlib inline
```

Every notebook in this course opens with a Setup cell that does exactly this. Copy it to your own projects — it's seven lines that save you hours of "why isn't my edit taking effect?"

Course-specific landmines — and fixes

Traps you'll meet in later notebooks

- **CVXPYgen (NB 3)** wants a working cmake and a C compiler. On macOS: `xcode-select --install`; on Linux: `apt install build-essential cmake`; on Windows: install Visual Studio Build Tools.
- **IPOPT + Pyomo (NB 4)** need compiled binaries. The reliable install is `conda install -c conda-forge ipopt cyipopt pyomo`. `pip install ipopt` usually fails.
- **Gurobi / MOSEK (NB 5, 13)** need a license file. Academic licenses are free but require registration through the vendor. The solver will import without the license; it just refuses to solve problems above a size cutoff.
- **ortools / PDLP (NB 7)** has version conflicts with some CVXPY releases. If you see “Unrecognized new version of ortools,” pin one: `pip install ortools==9.14.x`.

Good diagnostic habits

When a notebook fails at the *import* stage, the error almost always traces back to (a) wrong env active, (b) wrong kernel selected, or (c) mixed pip / conda install. Check these three before debugging the code.

Roadmap

Part 0.5 — Python Toolchain: Packages, Environments, Conventions

Part I — Modeling Languages and Solver Interfaces

Notebook 1 — SciPy optimize

Notebook 2 — CVXPY

Notebook 3 — CVXPYgen

Notebook 4 — IPOPT

Notebook 5 — Gurobi

Notebook 6 — SCIP

Notebook 7 — PDLP / PDHG

Notebook 8 — SQP

Notebook 15 — Benchmark Showdown

Notebook 16 — Debugging Cookbook

Notebook 17 — Modeling Tricks

Part II — Implement It Yourself

Part III — Large-Scale Case Studies

Deep Dive — Acceleration, Momentum, and Stochastic Gradient

Summary

Notebook 1: SciPy — The Swiss Army Knife

Maps to Course Notes Ch. 25.

Why start here?

- Ships with Python. No license, no install. Zero barrier.
- Covers line-search BFGS, trust-region Newton, SLSQP, `linprog`, `milp`.
- Every practitioner's first tool, every algorithm textbook's reference.

What you'll learn

- Extract iteration history through `callback=`
- Why a sparse Hessian is worth $100\times$ speedup
- SLSQP for small constrained NLPs; `trust-constr` for mid-sized
- A decision tree: which `method=` for which problem

Notebook 1 — The minimal example

```
from scipy import optimize
from ise5406_utils import rosenbrock, rosenbrock_grad, rosenbrock_hess

x0 = np.array([-1.2, 1.0])
res = optimize.minimize(
    rosenbrock, x0,
    jac=rosenbrock_grad, hess=rosenbrock_hess,
    method="trust-ncg",
    options={"xtol": 1e-10},
)
print(res.x, res.nit, res.nfev)
```

Pattern. Always pass analytical jac when you have it. Only pass hess if it's cheap — for big n , L-BFGS often beats Newton even with a free Hessian, because each iteration costs $O(n)$ vs. $O(n^3)$.

Notebook 1 — Scaling experiment

Sparse vs. dense Hessian on extended Rosenbrock

n	dense time (s)	sparse time (s)	speedup
100	0.03	0.02	1.5×
1,000	1.6	0.08	20×
10,000	>60 (memory)	1.1	>55×

At $n = 10,000$ the dense Hessian would be an 800 MB matrix. The sparse version stores $3n$ nonzeros. *Problem structure, not a clever algorithm, is the win here.*

Takeaway: Always tell SciPy about sparsity via `jac_sparsity=` and `hess_sparsity=`. Free speedup.

Notebook 1 — SciPy field guide

Entry points you should know

Function	When to use
<code>optimize.minimize(f, x0, ...)</code>	General unconstrained / box / NLP.
<code>optimize.least_squares(r, x0)</code>	Nonlinear LS; pass <code>jac_sparsity=</code>
<code>optimize.linprog(c, A_ub, b_ub, A_eq, b_eq, bounds)</code>	LP; prefer <code>method="highs"</code> .
<code>optimize.milp(c, ..., integrality=)</code>	Mixed-integer LP (HiGHS backend).
<code>optimize.root(F, x0)</code>	Solve $F(x) = 0$; Newton/Krylov variants.
<code>optimize.minimize_scalar</code>	1-D minimization (bracketing).

Instantiation checklist for minimize

- `method=` $\in$ `{"BFGS", "L-BFGS-B", "Newton-CG", "trust-ncg", "trust-constr", "SLSQP"}`
- `jac=` analytic gradient (or `True` if `f` returns $(f, \nabla f)$)
- `hess=` analytic Hessian; or `hessp=` a Hessian-vector product
- `bounds=`, `constraints=[LinearConstraint(...), NonlinearConstraint(...)]`
- `callback=lambda xk: hist.append(xk)` to record iterates
- `options={"maxiter":..., "gtol":..., "disp":True}`

Reading the OptimizeResult

`res.x` solution, `res.fun` optimum, `res.success` bool, `res.status/res.message`, `res.nit/res.nfev/res.njev`, `res.jac` gradient at solution. **Always check** `res.success` — a returned `x` that failed is just garbage with a number next to it.

Notebook 2: CVXPY — Disciplined Convex Programming

Maps to Course Notes Ch. 26.

Why CVXPY?

If SciPy is the Swiss Army knife, CVXPY is the laser-cut precision instrument. Write the problem *the way it appears on the lecture slide* and CVXPY handles the conic reformulation automatically.

The catch: DCP

You must stay inside **Disciplined Convex Programming** rules — a syntactic check that guarantees convexity. CVXPY will *refuse* to compile anything it cannot prove convex.

The refusal is a feature: 90% of “my optimization problem is weird” bugs are actually non-convex formulations that CVXPY would catch.

Notebook 2 — LASSO as DCP

LASSO problem. Given data $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, solve

$$\min_{x \in \mathbb{R}^n} \underbrace{\|Ax - b\|_2^2}_{\text{loss: convex quadratic}} + \underbrace{\lambda \|x\|_1}_{\text{sparsity regularizer}}, \quad \lambda \geq 0.$$

Each term is convex, so the sum is convex. The ℓ_1 term promotes *sparsity* — most entries of x^* are driven to exactly zero.

```
import cvxpy as cp

x = cp.Variable(n)
lam = cp.Parameter(nonneg=True, value=0.1)
loss = cp.sum_squares(A @ x - b) # ||Ax-b||_2^2
reg = lam * cp.norm1(x) # lambda * ||x||_1
prob = cp.Problem(cp.Minimize(loss + reg))
prob.solve(solver=cp.CLARABEL)

print("obj =", prob.value)
print("nnz =", (np.abs(x.value) > 1e-6).sum())
```

Code ↔ math

- $\text{cp.sum_squares}(A @ x - b) = \|Ax - b\|_2^2$
- $\text{cp.norm1}(x) = \|x\|_1 = \sum_i |x_i|$
- $\text{lam} = \lambda$ (a Parameter, so you can sweep it without re-parsing)
- $\text{Minimize}(\text{loss} + \text{reg})$ = the full objective above

What happens under the hood

CVXPY → canonicalize to conic standard form → dispatch to Clarabel / ECOS / SCS / MOSEK. Every solver swap is a one-line change.

Notebook 2 — Markowitz + dual variables

Extracting shadow prices

After `prob.solve()`, the dual variable of each constraint is available as `constraint.dual_value`. This *is* the Lagrange multiplier λ^* from the KKT system — no extra work.

Check Your Understanding (Q1, T/F)

“`cp.norm(A @ x - b, 2)**2` and `cp.sum_squares(A @ x - b)` are equivalent from CVXPY's perspective.”

Answer

Functionally yes, but the atom form `sum_squares` gives CVXPY more structure to exploit when emitting solver input. Prefer single atoms whenever possible.

Notebook 2 — CVXPY field guide

Building blocks

Object	Notes
<code>cp.Variable(n, nonneg=True, name=)</code>	Scalar/vector/matrix shapes; <code>integer=True</code> for MIP.
<code>cp.Parameter(shape, nonneg=, value=)</code>	Mutable data; required for DPP / codegen.
<code>cp.Constant(v)</code>	Wrap numpy arrays (rarely needed).
<code>cp.Minimize / cp.Maximize(expr)</code>	Objective wrapper.
<code>cp.Problem(obj, constraints)</code>	The problem; <code>is_dcp()</code> , <code>is_dpp()</code> verify.

Atoms you'll reach for

`sum_squares`, `norm(,1|2|"inf"|"fro")`, `quad_form(x, P)`, `log_sum_exp`, `entr`, `huber`, `max`, `min`, `pos`, `neg`, `abs`, `log_det`, `matrix_frac`, `power(x, p)`, `geo_mean`. *DCP rule:* use a single atom whenever possible — it exposes structure to the solver.

Solve & extract

`prob.solve(solver=cp.CLARABEL, verbose=True, warm_start=True)`; pick from `CLARABEL`, `ECOS`, `SCS`, `OSQP`, `MOSEK`, `GUROBI`, `CBC`.

Then: `x.value` (primal), `prob.value` (objective), `constraint.dual.value` (KKT multiplier), `prob.status` $\in$ {`OPTIMAL`, `OPTIMAL_INACCURATE`, `INFEASIBLE`, `UNBOUNDED`, `INFEASIBLE_OR_UNBOUNDED`, `SOLVER_ERROR`}.

`x.value` is `None` when the solve failed. Check `prob.status` *before* reading results; `OPTIMAL_INACCURATE` often means rescale the data or tighten tolerances.

Notebook 3: CVXPYgen — Generated Code for Parametric QPs

Extends Ch. 26.

The failure mode CVXPYgen fixes

CVXPY *reparses* the problem on every `solve()`. When you solve the same problem shape thousands of times (MPC, online learning, a real-time controller) parsing overhead dominates.

The fix

CVXPYgen emits a specialized C solver for a *parametric* problem. Parameters change between solves; structure does not. Only DPP-compliant problems (a strict subset of DCP) can be compiled.

Notebook 3 — MPC speedup

```
from cvxpygen import cpg

# x0 is the only thing that changes each step
x0 = cp.Parameter(nx, name="x0")
# ... build MPC problem using x0 ...
prob = cp.Problem(cp.Minimize(obj), constraints)

cpg.generate_code(prob, code_dir="mpc_gen", solver="OSQP")

# Now: thousands of solves, no parsing
for t in range(T):
    x0.value = current_state
    prob.solve(method="CPG")
    apply_control(u.value[:, 0])
```

Scaling experiment result

100–1000× speedup on repeated solves of a 10-state, 10-horizon MPC problem. The parse cost is amortized to zero; the remaining cost is raw QP solve time.

Notebook 3 — Deployment: Day 1 (offline codegen)

"Generate once, ship the folder, solve many times."

```
# --- Day 1: in your dev environment, one-shot ---
from cvxpygen import cpg
import pickle, os

# Everything that will change at runtime MUST be a Parameter
A = cp.Parameter((m, n), name="A")
b = cp.Parameter(m, name="b")
lam = cp.Parameter(nonneg=True, name="lam")
x = cp.Variable(n)
prob = cp.Problem(cp.Minimize(cp.sum_squares(A @ x - b) + lam*cp.norm1(x)))
assert prob.is_dpp() # stricter than DCP -- required for codegen

cpg.generate_code(prob, code_dir="lasso_solver", solver="OSQP")

with open("lasso_solver/problem.pickle", "wb") as f:
    pickle.dump(prob, f) # lets Day 2 reconstruct symbolic parameters
```

Commit the whole `lasso_solver/` directory into your repo. It contains the C source (`c/`), a `CMakeLists.txt`, and a compiled extension module. *No CVXPY is needed to run it later.*

Notebook 3 — Deployment: Day 2 (online solves)

```
# --- Day 2 (or Day 300): a fresh Python, a fresh server ---
import pickle, numpy as np
from lasso_solver.cpg_solver import cpg_solve

with open("lasso_solver/problem.pickle", "rb") as f:
    prob = pickle.load(f)
    prob.register_solve("CPG", cpg_solve)

while True:
    A_new, b_new = sensor.read()
    prob.param_dict["A"].value = A_new
    prob.param_dict["b"].value = b_new
    prob.param_dict["lam"].value = 0.05
    prob.solve(method="CPG", updated_params=["A", "b", "lam"])
    act_on(prob.var_dict["x"].value)
```

What is frozen at Day 1: problem dimensions, sparsity pattern, solver choice, pinned cvxpygen version.
Only parameter values change at Day 2. If n changes, regenerate.

Embedded path: skip Python at runtime entirely — the `c/` folder compiles stand-alone onto an STM32, an automotive ECU, or an FPGA. That is the whole point.

Notebook 4: IPOPT — The Gold Standard for NLPs

Maps to Course Notes Ch. 27.

What it is

Interior Point **OPT**imizer. Primal-dual interior point with logarithmic barrier, filter line search, second-order correction. Wächter & Biegler, open-source, decades mature.

When to reach for it

- Large, smooth, constrained NLP (up to $\sim 10^6$ variables)
- Convex or non-convex (local solution guarantee)
- You can supply analytic Jacobian and Hessian (or a sparsity pattern)

Three Python interfaces

pyomo (algebraic modeling), casadi (auto-diff + IPOPT bundled), cyipopt (low-level callbacks). All run the same C++ solver underneath.

Notebook 4 — HS071 benchmark in Pyomo

```
import pyomo.environ as pyo

m = pyo.ConcreteModel()
m.x = pyo.Var([1,2,3,4], bounds=(1, 5), initialize=1.0)
m.obj = pyo.Objective(
    expr = m.x[1]*m.x[4]*(m.x[1] + m.x[2] + m.x[3]) + m.x[3]
)
m.c1 = pyo.Constraint(expr = m.x[1]*m.x[2]*m.x[3]*m.x[4] >= 25)
m.c2 = pyo.Constraint(expr = sum(m.x[i]**2 for i in [1,2,3,4]) == 40)

pyo.SolverFactory("ipopt").solve(m, tee=True)
```

Jacobian sparsity matters. On a 5000-variable hanging chain, dense Jacobian = 12s; pattern-aware Jacobian = 0.3s. Same iterates.

Notebook 4 — IPOPT field guide

Reading the IPOPT log — quick reference

Column	Meaning	When to worry
inf_pr	Primal infeasibility	Not shrinking $\Rightarrow$ check model / scaling
inf_du	Dual infeasibility	Large $\Rightarrow$ bad gradients / ill-conditioned
lg(mu)	Log barrier param	Should decrease smoothly to $-\infty$
alpha_pr	Primal step length	Many $< 10^{-3} \Rightarrow$ bad search direction
ls	Line search trials	Values > 4 sustained $\Rightarrow$ stalling

“Solved to acceptable level” $\neq$ “solved”. Verify with a residual check on your model constraints before trusting the answer.

Notebook 4 — Pyomo + IPOPT field guide

Core Pyomo constructors

Object	Notes
<code>pyo.ConcreteModel()</code>	Instantiate; values bound now.
<code>pyo.Set(initialize=[...])</code>	Index sets for vars/constraints.
<code>pyo.Param(S, initialize=, mutable=True)</code>	Data; mutable lets you re-solve.
<code>pyo.Var(S, bounds=(l,u), initialize=, within=pyo.NonNegativeReals)</code>	Binary, Integers, Reals, ...
<code>pyo.Objective(expr=, sense=pyo.minimize)</code>	One per model (or deactivate).
<code>pyo.Constraint(S, rule=lambda m,i: ...)</code>	Or <code>expr=</code> for single.
<code>pyo.ConstraintList()</code>	Imperative build; <code>m.c.add(...)</code> .

Solving with IPOPT

```
solver = pyo.SolverFactory("ipopt")
solver.options["tol"] = 1e-8
solver.options["max_iter"] = 3000
solver.options["linear_solver"] = "mumps" # or ma27/ma57 if HSL installed
results = solver.solve(m, tee=True)
```

Reading results

`pyo.value(m.x[i])` for primal values, `m.obj()` for the objective, `m.dual[m.c1]` after `m.dual = pyo.Suffix(direction=pyo.Suffix.IMPORT)`. Status: `results.solver.status` $\in$ {ok, warning, error}; `results.solver.termination_condition` $\in$ {optimal, infeasible, maxIterations, locallyOptimal, userInterrupt}.

IPOPT returns *local* optima for non-convex NLPs. Multi-start from different `initialize=` values, or use `bonmin/couenne` for guaranteed global on smaller models.

Notebook 5: Gurobi — Industrial MIP and QP

Maps to Course Notes Ch. 28.

What sets Gurobi apart

State-of-the-art heuristics, presolve, cut generation, branching, warm starts — all tuned over twenty-plus years. For MILP in particular it is typically **10–100× faster than any open-source alternative**.

What it handles

LP, QP, MIP, MIQP, MIQCP, non-convex QCQP (via spatial B&B), warm starts, callback-style lazy constraints, multi-threaded.

Free academic license. The notebooks fall back to HiGHS / Clarabel / SCIP if Gurobi is unavailable, so the code runs for everyone.

Notebook 5 — Facility location MIP

```
import gurobipy as gp
from gurobipy import GRB

m = gp.Model()
y = m.addVars(n_fac, vtype=GRB.BINARY, name="open")
x = m.addVars(n_fac, n_cust, vtype=GRB.CONTINUOUS, name="flow")

m.setObjective(
    gp.quicksum(f[j]*y[j] for j in range(n_fac))
    + gp.quicksum(c[j,i]*x[j,i] for j in range(n_fac) for i in range(n_cust)),
    GRB.MINIMIZE)
m.addConstrs(gp.quicksum(x[j,i] for j in range(n_fac)) == 1
              for i in range(n_cust))
m.addConstrs(x[j,i] <= y[j]
              for j in range(n_fac) for i in range(n_cust))
m.optimize()
```

The single biggest lever: warm starts

For sequential MIPs (optimization of a time-varying plan), set `var.Start = previous_value` to cut solve time by 3–10×. The notebook benchmarks this.

Notebook 6: SCIP — Open-Source MINLP

Maps to Course Notes Ch. 28.

SCIP's niche

MINLP. Mixed-integer *nonlinear* programming — polynomial constraints, exp/log/trig — with a global guarantee via spatial branch-and-bound. Gurobi does MIP and non-convex QCQP; SCIP goes further into general MINLP.

Why it matters

- Academic license for commercial use (free for research / teaching)
- Plugin architecture — write custom constraint handlers
- The only fully open-source solver that handles true MINLP

Check Your Understanding (Q1, T/F)

“SCIP's spatial branch-and-bound for MINLP is guaranteed to find the global optimum given enough time.”

Answer

True — in the limit. SCIP tightens bounds using convex/concave envelopes on a fixed operator library. Practical performance depends on how tight those envelopes are.

Notebook 7: PDLP and PDHG — First-Order Methods for Huge LPs

Maps to Course Notes Ch. 23.

The wall that classical methods hit

For LPs with *millions* of variables, simplex's basis becomes unwieldy and interior-point methods need to factor a dense normal-equations matrix whose memory scales like $O(n^2)$.

First-order methods break the wall

PDHG (Chambolle–Pock): saddle-point reformulation $\min_x \max_y \{c^\top x + y^\top (Ax - b)\}$ — alternating proximal steps, *matrix-vector products only*.

PDLP (Google OR-Tools)

Production version of PDHG. Used for LP relaxations with 10^7+ nonzeros at Google. The notebook solves a 10M-nonzero transportation LP that simplex and IPM cannot fit in memory.

Notebook 7 — PDHG from scratch

```
def pdhg(c, A, b, sigma, tau, n_iter):  
    x = np.zeros(A.shape[1]); y = np.zeros(A.shape[0])  
    xbar = x.copy()  
    for k in range(n_iter):  
        y = y + sigma * (A @ xbar - b)           # dual step  
        x_new = np.maximum(0, x - tau * (c + A.T @ y)) # primal step + proj  
        xbar = 2*x_new - x                        # extrapolation  
        x = x_new  
    return x, y
```

Step-size condition: $\sigma\tau\|A\|_2^2 < 1$. Violate it and iterates diverge (often silently for hundreds of iterations, then blow up).

Scaling experiment result

PDHG on a sparse transportation LP with 10 M nonzeros: 90s wall time on a laptop. The same problem crashes both Clarabel (IPM) and HiGHS simplex on an OOM error.

Notebook 8: SQP — The Underrated Workhorse

Expands Ch. 21.7.4.

What is SQP really?

Newton's method applied to the KKT system. At each iterate solve a local QP:

$$\min_p \frac{1}{2} p^\top \nabla^2 \mathcal{L}_k p + \nabla f_k^\top p \quad \text{s.t.} \quad \nabla h_k^\top p + h_k = 0$$

The solution is the Newton step on the primal-dual KKT equations.

When SQP wins

- Medium size, very smooth, many equality constraints
- Expensive function evaluations (aerospace, chemistry)
- You want *quadratic* local convergence with explicit constraints

SciPy's `method="SLSQP"` is a reference SQP implementation. It hits the solution of a nonlinear hanging chain in 12 iterations where steepest-descent projected gradient would take thousands.

Notebook 8 — Minimal SQP in 25 lines

```
def sqp_equality(f, grad_f, hess_f, h, grad_h, x0, n_iter=50, tol=1e-8):
    x = x0.copy()
    lam = np.zeros(h(x).size)
    for k in range(n_iter):
        g = grad_f(x); H = hess_f(x); hv = h(x); A = grad_h(x)
        # KKT block matrix: solve for (p, lam_new)
        KKT = np.block([[H, A.T], [A, np.zeros((A.shape[0], A.shape[0]))]])
        rhs = -np.concatenate([g + A.T @ lam, hv])
        sol = np.linalg.solve(KKT, rhs)
        p, dlam = sol[:x.size], sol[x.size:]
        x = x + p
        lam = lam + dlam
        if np.linalg.norm(p) < tol: break
    return x, lam
```

Big-picture contrast

IPM: stay interior, one system solve per iteration, barrier $\mu \rightarrow 0$. **SQP:** linearize KKT, one QP per iteration, constraints handled exactly.

Notebook 15: Solver Benchmark Showdown

Capstone for Part I.

What we benchmark

Same problems across many solvers. Six problem classes at multiple sizes, each run many times, scored by wall time and solution quality.

Problem classes

- LP — transportation, netgen instances
- QP — Markowitz portfolio, varying covariance structure
- MILP — knapsack (hard), facility location
- NLP — hanging chain (equality constrained)
- LASSO — large convex sparse

The output: Dolan–Moré performance profiles

$\rho_s(\tau)$ = fraction of problems solver s solves within τ of the fastest. Curves near the left-top corner = fast and robust.

Notebook 15 — Why performance profiles?

Why not just “mean time”?

Mean is dominated by slow outliers. A solver that's $10\times$ slower on 10% of instances but $2\times$ faster on the rest has a bad mean, but may be the right choice if you care about median-case behavior.

What the profile separates

- *Typically good*: height of the curve at $\tau = 1$
- *Never catastrophically bad*: height of the curve at $\tau = \infty$

If a solver's curve reaches 1.0 only at $\tau = 5$, it means that solver *did* solve every instance — but sometimes $5\times$ slower than the best.

Notebook 16: Debugging & Diagnostics Cookbook

Reference notebook.

The idea

Every failure mode is *deliberately induced* — infeasibility, unbounded, DCP rejection, indefinite BFGS Hessian, Gurobi numerical issues, saddle points — and the diagnostic walked through end-to-end.

Representative Q&A

Q. Your LP solver reports a “dual infeasibility certificate”. What do you do?

A. LP duality: dual infeasible $\Leftrightarrow$ primal either infeasible or unbounded. Solve a feasibility LP (drop the objective) to distinguish.

Bookmark this notebook. When your real optimization problem misbehaves, the error message is usually one of the eight categories here.

Notebook 17: Modeling Tricks Catalog

Reference notebook.

Why this exists

Good optimization modeling is half theorem-proving, half folklore. This notebook is the folklore, made explicit.

Trick families

- Nonsmooth $\rightarrow$ smooth: $\max(x, 0)$, $|x|$, $\max_i x_i$, minimax
- Logical $\rightarrow$ big-M: if-then constraints, disjunctions
- Cardinality $\|x\|_0 \leq k$ via binary z_i with $|x_i| \leq Mz_i$
- Products of variables $\rightarrow$ McCormick or trilinear envelopes
- Fractional objectives $\rightarrow$ Charnes–Cooper transformation
- Norm constraints $\rightarrow$ conic reformulations

Notebook 17 — The ReLU Trick

Model $z = \max(x, 0)$ in a convex program

Introduce $z \geq 0$ and $z \geq x$. Then $z \geq \max(x, 0)$ by construction. In minimization, z is driven down to *exactly* $\max(x, 0)$.

Why it works

Any convex objective coupled with $z \geq \max(x, 0)$ is minimized when z equals the lower bound, so the extra slack is automatically eliminated. The “slackness” is a feature of optimality, not a modeling bug.

Big-M pitfall. Choosing $M = 10^6$ “to be safe” cripples the LP relaxation. Use the tightest valid M , or indicator constraints (Gurobi) / SOS1 (SCIP) instead.

Roadmap

Part 0.5 — Python Toolchain: Packages, Environments, Conventions

Part I — Modeling Languages and Solver Interfaces

Part II — Implement It Yourself

Notebook 9 — Line Search from Scratch

Notebook 10 — CG / PCG

Notebook 11 — Trust Region

Notebook 12 — BFGS / L-BFGS

Notebook 18 — Projected Gradient / Frank-Wolfe

Notebook 19 — ISTA / FISTA

Notebook 20 — ADMM

Notebook 22 — Hidden III-Conditioning

Notebook 23 — SGD / Momentum / Adam

Part III — Large-Scale Case Studies

Deep Dive — Acceleration, Momentum, and Stochastic Gradient

Summary

Notebook 9: Line Search Methods from Scratch

Maps to Course Notes Ch. 12, 13.

The template every line search method shares

$$x^{k+1} = x^k + \alpha_k p^k$$

choosing p^k (direction) and α_k (step). This notebook implements three α_k strategies on three p^k directions and tests them on Rosenbrock.

Strategies implemented

- **Backtracking Armijo** — the workhorse. Simple, cheap, reliable.
- **Strong Wolfe** — enforces sufficient decrease *and* curvature. Nocedal–Wright “zoom” algorithm.
- **Exact minimization** — only for quadratics, where $\alpha^* = -g^\top p / p^\top H p$.

Notebook 9 — Backtracking Armijo

```
def armijo(f, grad, x, p, alpha0=1.0, c1=1e-4, rho=0.5, max_iter=50):  
    """Backtracking line search: find alpha with sufficient decrease."""  
    fx = f(x); gx = grad(x)  
    descent = gx @ p  
    assert descent < 0, "p must be a descent direction"  
    alpha = alpha0  
    for _ in range(max_iter):  
        if f(x + alpha*p) <= fx + c1*alpha*descent:  
            return alpha  
        alpha *= rho  
    return alpha    # return whatever; convergence theorem still holds
```

Why $c_1 = 10^{-4}$?

Very loose: we ask for only 10^{-4} of the predicted decrease. Near the optimum the quadratic term matters, and the best step gives exactly the predicted decrease — tighter c_1 would reject it.

Notebook 9 — A pathological example

The Dennis–Schnabel counterexample

A 2-D problem where pure steepest descent with constant step diverges, but steepest descent with Armijo backtracking converges. The backtracking is what rescues the algorithm.

Take-away

Line search turns a “method that may or may not work” into a *globally convergent* method. It costs a few extra function evaluations per iteration; it is worth every one.

Check Your Understanding

Q. Why not set $c_1 = 0.5$? **A.** Too strong. Near the optimum the quadratic model predicts exactly the decrease a good step produces. $c_1 = 0.5$ would reject good steps.

Notebook 10: Conjugate Gradient from Scratch

Maps to Course Notes Ch. 17.

What CG solves

$\min_x \frac{1}{2}x^\top Ax - b^\top x$ for symmetric positive-definite A . Equivalently: $Ax = b$.

What makes CG special

- Uses *only* matrix-vector products Av — no factoring, no explicit A required.
- Converges in $\leq n$ iterations exactly (finite termination).
- In practice: $O(\sqrt{\kappa})$ iterations to precision ϵ — *dramatically* faster than steepest descent's $O(\kappa)$.

“Matrix-free” is the magic phrase. CG is the default sub-solver in large-scale trust-region Newton (Steihaug) precisely because the Hessian is only available through $\nabla^2 f(x) \cdot v$ products.

Notebook 10 — CG and PCG

```
def cg(A, b, x0=None, tol=1e-8, max_iter=None):
    n = b.size
    x = np.zeros(n) if x0 is None else x0.copy()
    r = b - A @ x
    p = r.copy()
    rs_old = r @ r
    for k in range(max_iter or n):
        Ap = A @ p
        alpha = rs_old / (p @ Ap)
        x += alpha * p
        r -= alpha * Ap
        rs_new = r @ r
        if np.sqrt(rs_new) < tol: break
        p = r + (rs_new / rs_old) * p
        rs_old = rs_new
    return x, k+1
```

Scaling experiment

PCG vs. dense solve on the 2-D Poisson problem at grid size $N \times N$: at $N = 512$, dense solve ~ 100 s; PCG with incomplete-Cholesky preconditioner ~ 0.8 s.

Notebook 11: Trust Region Methods from Scratch

Maps to Course Notes Ch. 16.

Line search vs. trust region

- Line search fixes a direction, searches for a step length
- Trust region fixes a region, solves a constrained subproblem

The trust-region subproblem

$$\min_p m_k(p) := f_k + g_k^\top p + \frac{1}{2} p^\top B_k p \quad \text{s.t.} \quad \|p\|_2 \leq \Delta_k$$

Two classical subsolvers:

- **Dogleg** — exact when B_k is PD; needs an explicit Newton step
- **Steihaug–Toint CG** — matrix-free, works for indefinite B_k

Notebook 11 — The trust-region update rule

Core idea

Actual reduction / predicted reduction = ρ_k . Trust the model more or less based on how accurate it was.

$\rho_k < 0.25$	shrink: $\Delta_{k+1} = 0.25\Delta_k$
$\rho_k > 0.75$ & boundary hit	grow: $\Delta_{k+1} = 2\Delta_k$
otherwise	$\Delta_{k+1} = \Delta_k$

$\rho_k < 0$	reject the step; $x^{k+1} = x^k$
--------------	----------------------------------

Check Your Understanding

Q. Your trust-region code reports $\rho = -0.5$. What happened?

A. The step *increased* f , though the model predicted a decrease. Step is rejected; Δ shrinks to 0.25Δ . Next iteration the smaller region should produce a trustworthy model.

Notebook 12: BFGS and L-BFGS from Scratch

Maps to Course Notes Ch. 15.

What quasi-Newton methods do

Approximate $\nabla^2 f$ using only gradient pairs $(s_k, y_k) = (x^{k+1} - x^k, g^{k+1} - g^k)$.

BFGS update

$$H_{k+1} = (I - \rho_k s_k y_k^\top) H_k (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top, \quad \rho_k = 1/(y_k^\top s_k)$$

Maintains PD, has superlinear local convergence.

L-BFGS = BFGS with bounded memory

Store only the last m pairs (s_i, y_i) . Apply H -action via the *two-loop recursion* in $O(mn)$ per iteration. No $n \times n$ matrix ever materialized.

Notebook 12 — L-BFGS two-loop recursion

```
def lbfgs_direction(g, s_list, y_list, gamma):  
    """Compute  $-H_k * g$  using the L-BFGS two-loop recursion."""  
    q = g.copy()  
    alpha = np.empty(len(s_list))  
    # First loop: backward through history  
    for i in reversed(range(len(s_list))):  
        rho = 1.0 / (y_list[i] @ s_list[i])  
        alpha[i] = rho * (s_list[i] @ q)  
        q -= alpha[i] * y_list[i]  
    r = gamma * q                                #  $H_0 * q$   
    # Second loop: forward through history  
    for i in range(len(s_list)):  
        rho = 1.0 / (y_list[i] @ s_list[i])  
        beta = rho * (y_list[i] @ r)  
        r += (alpha[i] - beta) * s_list[i]  
    return -r                                    # search direction
```

The storage story

At $n = 10,000$, BFGS needs an 800 MB matrix; L-BFGS with $m = 10$ needs ~ 1.6 MB. Same progress per iteration on smooth problems.

Notebook 18: Projected Gradient & Frank-Wolfe

Maps to Course Notes Ch. 21.3–21.4.

Both solve

$\min_{x \in \mathcal{C}} f(x)$ for closed convex $\mathcal{C}$, smooth f . They differ in *what they ask per iteration*.

Method	Per-iteration work	When it wins
Projected gradient	$P_{\mathcal{C}}(x - \alpha \nabla f)$	projection is cheap
Frank–Wolfe	$\arg \min_{s \in \mathcal{C}} \nabla f^{\top} s$	linear oracle is cheap

Sparsity by design. Frank–Wolfe iterates are convex combinations of at most $k + 1$ extreme points after k iterations. For an ℓ_1 ball or nuclear-norm ball, the iterate is automatically k -sparse or rank- k .

Notebook 18 — Projected gradient in 10 lines

```
def projected_gradient(f, grad, proj, x0, alpha=1e-2, n_iter=500, tol=1e-8):
    x = x0.copy()
    hist = []
    for k in range(n_iter):
        g = grad(x)
        x_new = proj(x - alpha * g)
        hist.append(f(x))
        if np.linalg.norm(x_new - x) < tol: break
    x = x_new
    return x, hist
```

Common projections

$P_{[a,b]}$ = clip; $P_{\mathbb{R}_+^n}$ = clip at 0; P_{Δ} (simplex) = sort + threshold; $P_{\ell_2}(x) = x \cdot \min(1, R/\|x\|)$. Each is $O(n)$ or $O(n \log n)$. Box and simplex projections are the everyday workhorses.

Notebook 19: Proximal Gradient — ISTA and FISTA

Maps to Course Notes Ch. 18.3.

Composite optimization

$$\min_x f(x) + g(x)$$

where f is smooth and g is nonsmooth but has a cheap **proximal operator**

$$\text{prox}_{\alpha g}(z) = \arg \min_v g(v) + \frac{1}{2\alpha} \|v - z\|_2^2.$$

ISTA: plain proximal gradient

$$x^{k+1} = \text{prox}_{\alpha g}(x^k - \alpha \nabla f(x^k))$$

$O(1/k)$ rate. For $g = \lambda \|\cdot\|_1$, the prox is element-wise *soft threshold*.

FISTA: Nesterov acceleration

Add an extrapolation step $y^k = x^k + \frac{k-1}{k+2}(x^k - x^{k-1})$, apply prox at y^k . Rate jumps to $O(1/k^2)$. Same per-iteration cost.

Notebook 19 — FISTA in 15 lines

```
def fista(A, b, lam, L, x0=None, n_iter=500, tol=1e-8):
    n = A.shape[1]
    x_prev = x0 if x0 is not None else np.zeros(n)
    y = x_prev.copy()
    t = 1.0
    hist = []
    for k in range(n_iter):
        grad = A.T @ (A @ y - b)
        x = soft_threshold(y - grad/L, lam/L)
        t_new = 0.5 * (1 + np.sqrt(1 + 4*t*t))
        y = x + ((t - 1)/t_new) * (x - x_prev)
        t, x_prev = t_new, x
        hist.append(0.5*np.linalg.norm(A@x-b)**2 + lam*np.linalg.norm(x,1))
    return x, hist

def soft_threshold(z, t):
    return np.sign(z) * np.maximum(np.abs(z) - t, 0.0)
```

Scaling experiment

On LASSO with $n = 10,000$, $m = 2,000$: ISTA reaches 10^{-4} suboptimality in 1200 iterations; FISTA in 85. The $O(1/k^2)$ vs. $O(1/k)$ is *visibly an order of magnitude*.

Notebook 20: ADMM and Augmented Lagrangian

Maps to Course Notes Ch. 21.6.

ADMM solves

$$\min_{x,z} f(x) + g(z) \quad \text{s.t.} \quad Ax + Bz = c$$

by alternating primal updates and a dual ascent on the multiplier of the *augmented* Lagrangian (adds a $(\rho/2)\|\cdot\|^2$ penalty).

Why it's the workhorse of distributed optimization

Each x - or z -update is a *separate* proximal subproblem. If f and g decouple by components, each subproblem is embarrassingly parallel. This is **consensus ADMM**.

Notebook 20 — ADMM on LASSO

```
def admm_lasso(A, b, lam, rho=1.0, n_iter=500):
    n = A.shape[1]
    x = np.zeros(n); z = np.zeros(n); u = np.zeros(n)
    # Cache the  $(A.T A + \rho I)^{-1}$  factor once
    AtA_rho = A.T @ A + rho * np.eye(n)
    L = np.linalg.cholesky(AtA_rho)
    Atb = A.T @ b
    for k in range(n_iter):
        # x update: quadratic - single factor reuse
        rhs = Atb + rho * (z - u)
        x = np.linalg.solve(L.T, np.linalg.solve(L, rhs))
        # z update: soft threshold
        z = soft_threshold(x + u, lam/rho)
        # dual update
        u = u + x - z
    return z
```

Increasing ρ speeds primal convergence but slows dual convergence. Adaptive schemes (Boyd et al., §3.4.1) adjust ρ online.

Notebook 22: Hidden Ill-Conditioning

Diagnostic skills.

The trap

A problem formulation looks clean. The code runs without errors. A solver returns an answer — and the answer is *catastrophically wrong*, by many orders of magnitude. Nobody notices.

Three examples dissected

- **Structural beam:** stiffness matrix condition grows as L^4
- **Polynomial regression:** Vandermonde matrix condition grows exponentially in degree
- **Portfolio:** sample covariance where $m < n$ is singular up to floating-point noise

Diagnostic checklist: compute $\kappa(A)$, compute eigenvalue ratio, check that $\|Ax - b\|$ is small *and* $\kappa \cdot \|Ax - b\| / \|b\|$ is small.

Notebook 22 — Quick check

Check Your Understanding

Q. You solve $Ax = b$ and observe $\|Ax - b\| = 10^{-12}$. Do you trust x ?

A. Not without knowing $\kappa(A)$. The forward error satisfies

$$\frac{\|x - x^*\|}{\|x^*\|} \leq \kappa(A) \cdot \frac{\|Ax - b\|}{\|b\|}.$$

If $\kappa = 10^8$, x could be off by 10^{-4} in relative terms despite the tiny residual.

Preconditioning saves the day

Cholesky preconditioning on the Poisson / beam system makes PCG converge in **a single iteration** regardless of mesh size — “mesh-independent convergence.” The scaling experiment shows this empirically.

Notebook 23: SGD, Momentum, and Adam from Scratch

Maps to Course Notes Ch. 18.

The family tree

- **GD**: $x^{k+1} = x^k - \alpha \nabla f(x^k)$. Baseline; $O(\kappa)$ iterations.
- **Polyak heavy ball**: $x^{k+1} = x^k - \alpha g_k + \beta(x^k - x^{k-1})$. $O(\sqrt{\kappa})$.
- **Nesterov**: look-ahead gradient. Provably optimal first-order rate.
- **SGD**: replace ∇f with a stochastic estimate $\hat{g}_k$.
- **AdaGrad / RMSprop / Adam**: *per-parameter* adaptive step size.

Scaling experiment

On a condition-100 quadratic: GD needs 692 iterations, heavy-ball needs 48. The $O(\sqrt{\kappa})$ speedup is *visible as an order of magnitude*.

Notebook 23 — Adam in 10 lines

```
def adam_step(x, g, m, v, t, lr=1e-3, b1=0.9, b2=0.999, eps=1e-8):  
    m = b1 * m + (1 - b1) * g  
    v = b2 * v + (1 - b2) * (g * g)  
    m_hat = m / (1 - b1**t)           # bias correction  
    v_hat = v / (1 - b2**t)  
    x = x - lr * m_hat / (np.sqrt(v_hat) + eps)  
    return x, m, v
```

The intuition

Adam = momentum (first moment m_t) + diagonal preconditioning (inverse $\sqrt{v_t}$). It's “momentum plus Jacobi.”

Why Adam dominates in ML

Noisy gradients + per-parameter scale differences + poor conditioning = exactly Adam's design space. For smooth convex problems, Nesterov is often faster; for deep nets, Adam is remarkably robust.

Roadmap

Part 0.5 — Python Toolchain: Packages, Environments, Conventions

Part I — Modeling Languages and Solver Interfaces

Part II — Implement It Yourself

Part III — Large-Scale Case Studies

Notebook 13 — Portfolio Optimization

Notebook 14 — Neural Network Training

Notebook 21 — Compressed Sensing

Deep Dive — Acceleration, Momentum, and Stochastic Gradient

Summary

Notebook 13: Portfolio Optimization on Real Market Data

Maps to Course Notes Ch. 30.

The full pipeline

1. Download historical returns via `yfinance`
2. Estimate mean return $\hat{\mu}$ and covariance $\hat{\Sigma}$
3. Classical Markowitz: $\min_w w^\top \hat{\Sigma} w$ s.t. $\hat{\mu}^\top w = r_t, \mathbf{1}^\top w = 1$
4. **Cardinality constraint:** hold at most k stocks (requires MIP)
5. **Robust portfolio:** uncertainty set for $\hat{\mu}$
6. Out-of-sample backtest

Technical subtlety

`cp.psd.wrap( $\Sigma$ )` forces CVXPY to trust that Σ is PSD. Sample covariance is PSD in exact arithmetic but can show tiny negative eigenvalues in floating point. The wrap bypasses the DCP safety check.

Notebook 13 — Robustness matters

Why robust optimization?

Classical Markowitz on real market data is *extremely* sensitive to $\hat{\mu}$ — a 1% change in estimated return can move the allocation from 100% in stock A to 100% in stock B.

The robust formulation

$$\min_w w^\top \hat{\Sigma} w \quad \text{s.t.} \quad \min_{\mu \in \mathcal{U}} \mu^\top w \geq r_t, \quad \mathbf{1}^\top w = 1$$

For an ellipsoidal $\mathcal{U}$, the inner minimization has a closed form and the whole problem is an SOCP.

The out-of-sample backtest in the notebook shows: robust portfolios give up $\sim 5\%$ in realized return for $\sim 40\%$ lower drawdown. Free variance reduction.

Notebook 14: Neural Network Training as Optimization

Maps to Course Notes Ch. 31.

Training a neural net is optimization

- SGD = steepest descent with noisy gradient
- Adam = diagonally preconditioned momentum
- PyTorch's LBFGS = Notebook 12 algorithm applied to a 10^6 -variable problem

What is different about ML?

- Scale: parameters $n \sim 10^6$ – 10^9
- Stochastic objective: sample-mean approximation of an expectation
- Extremely non-convex, but implicit regularization from the method matters as much as from the penalty

Notebook 14 — The four optimizers side by side

MNIST MLP benchmark

Optimizer	test acc	wall time	learning-rate sensitivity
SGD (plain)	96.8%	1×	very high
SGD + momentum	97.9%	1×	high
Adam	98.1%	1.1×	low
L-BFGS	97.4%	0.7×	n/a (full batch)

Why we don't use Newton on neural nets

Hessian size $n^2 \sim 10^{12}$ for a 10^6 -parameter model. Even if it fit, it would be indefinite almost everywhere (saddle-point dominated landscape). Adam implicitly *uses* saddle structure rather than trying to escape.

Notebook 21: Compressed Sensing and Signal Recovery

Maps to Course Notes Ch. 34.

The theorem in one line

You can recover a k -sparse signal from $m = O(k \log(n/k))$ random measurements by solving a convex optimization problem.

Measurement model: $b = Ax^* + \epsilon$ with $A \in \mathbb{R}^{m \times n}$, $m \ll n$.

Two formulations

- **Basis pursuit** (noiseless): $\min \|x\|_1$ s.t. $Ax = b$.
- **LASSO** (noisy): $\min \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|x\|_1$.

Why ℓ_1 not ℓ_0 ? $\|\cdot\|_0$ is NP-hard; $\|\cdot\|_1$ is its tightest convex envelope on the unit cube. Under the Restricted Isometry Property, the relaxation is *tight*.

Notebook 21 — The Donoho–Tanner phase transition

Empirical experiment

Generate k -sparse signals, reconstruct from m random Gaussian measurements, for varying $(m/n, k/m)$. Count success rate over trials.

The phase transition

There is a *sharp* curve in $(m/n, k/m)$ space: below it, recovery always fails; above it, recovery always succeeds. The curve is predicted analytically and *matches experiment exactly*.

Why this matters

One of the most beautiful theory–experiment matches in modern applied math. It turns “CS works sometimes” into “CS works exactly when the sampling rate exceeds this explicit threshold.”

Practical rule of thumb: $m \approx 4k$ measurements reliably recover k -sparse signals for n up to 10^6 .

Roadmap

Part 0.5 — Python Toolchain: Packages, Environments, Conventions

Part I — Modeling Languages and Solver Interfaces

Part II — Implement It Yourself

Part III — Large-Scale Case Studies

Deep Dive — Acceleration, Momentum, and Stochastic Gradient

Summary

Motivation — gradient descent leaves a lot on the table

The benchmark: gradient descent on a smooth strongly convex f

For L -smooth, μ -strongly-convex f with condition number $\kappa = L/\mu$:

$$\text{GD: } f(x_k) - f^* \leq \left(\frac{\kappa-1}{\kappa+1}\right)^{2k} (f(x_0) - f^*)$$

Ill-conditioned problems ($\kappa \gg 1$): contraction rate $\rightarrow 1$. Convergence grinds to a halt. You see this live in Notebook 22.

Today's three ideas — each fixes a different failure

- **Momentum.** Low-frequency errors die slowly under plain GD. Add a velocity term; accumulate consistent descent directions.
- **Nesterov acceleration.** A *provably optimal* look-ahead version of momentum: $O(1/k^2)$ on smooth convex, $\sqrt{\kappa}$ -rate on strongly convex.
- **Stochastic gradient.** When the full ∇f is prohibitive (sum over millions of data points), sample an unbiased estimate. Different scaling regime entirely.

Heavy-ball momentum — Polyak, 1964

Physical intuition

A marble rolling down a bowl doesn't reverse direction every time the slope flips sign — its inertia carries it through small oscillations. Heavy-ball replaces

$$x_{k+1} = x_k - \alpha \nabla f(x_k) \quad \text{with} \quad x_{k+1} = x_k - \alpha \nabla f(x_k) + \beta (x_k - x_{k-1}).$$

The term $\beta(x_k - x_{k-1})$ is the *velocity* — a moving average of past steps.

```
# one heavy-ball step
v_new = beta * v - alpha * grad(x)
x_new = x + v_new
```

Why it works on ill-conditioned quadratics

In eigencoordinates of the Hessian, GD on a coordinate with eigenvalue λ contracts by $|1 - \alpha\lambda|$. Small eigenvalues contract slowly. Heavy-ball turns this into a *second-order* recursion; with the right (α, β) , contraction becomes $\sqrt{1 - \alpha\lambda}$ — a **square root of the GD rate**. For $\kappa = 10^4$, that's the difference between 10^4 iterations and 100.

Nesterov acceleration — momentum, but you evaluate where you're going

The two-line change from heavy-ball

Instead of

$$x_{k+1} = x_k - \alpha \nabla f(x_k) + \beta(x_k - x_{k-1}),$$

evaluate the gradient at the *extrapolated* point:

$$y_k = x_k + \beta(x_k - x_{k-1}), \quad x_{k+1} = y_k - \alpha \nabla f(y_k).$$

You look one step ahead along the momentum direction, then take a gradient step from there.

Why the lookahead matters

Heavy-ball can overshoot: momentum pushes you past the minimum, and the old gradient (evaluated before the push) doesn't know. Nesterov peeks at the destination and applies a *corrective* gradient — it's self-damping on the curves that used to oscillate.

Rates (Nesterov 1983; Beck–Teboulle FISTA 2009)

For L -smooth convex f : $f(x_k) - f^* \leq \frac{2L\|x_0 - x^*\|^2}{(k+1)^2}$ — *optimal* among first-order methods using only ∇f .

For strongly convex: linear rate $(1 - \sqrt{1/\kappa})^k$, beating GD's $(1 - 1/\kappa)^k$.

Nesterov — the standard t_k schedule

The momentum coefficient $\beta_k = (t_{k-1} - 1)/t_k$ grows with k , with t_k satisfying $t_{k+1}^2 = t_{k+1} + t_k^2$. This is the schedule that achieves the $O(1/k^2)$ rate; it's also what FISTA uses.

```
# Nesterov / FISTA inner loop
x_prev, y, t = x0.copy(), x0.copy(), 1.0
for k in range(max_iter):
    x_new = y - alpha * grad(y)                # gradient step from lookahead
    t_new = 0.5 * (1.0 + np.sqrt(1.0 + 4.0*t*t)) # Nesterov schedule
    beta = (t - 1.0) / t_new                    # momentum coefficient
    y     = x_new + beta * (x_new - x_prev)      # extrapolate for next iter
    x_prev, t = x_new, t_new
```

What to notice in the code

- Exactly one gradient eval per iteration — same budget as GD.
- Two state variables: the iterate x and the extrapolated y .
- $\beta_k \rightarrow 1$ as $k \rightarrow \infty$ — more momentum later, when you're oscillating near the optimum.

On NB 19 (LASSO/FISTA), this replaces the $O(1/k)$ ISTA rate by $O(1/k^2)$: the same 1200-iteration budget that got ISTA to 10^{-4} suboptimality gets FISTA to 10^{-6} .

Heavy-ball vs. Nesterov — when to prefer which

On smooth convex (and especially smooth-strongly-convex)

- **Nesterov** has a clean convergence proof and optimal rate. Use it whenever you trust L -smoothness (or can use a line search).
- **Heavy-ball** has no such guarantee on general convex problems; its rate requires careful (α, β) tuning and is brittle. It *can* beat Nesterov in practice on some quadratic problems but the theory doesn't cover non-quadratics.

On non-convex problems (deep learning)

- Theory breaks down either way; both are heuristics.
- Empirically, PyTorch / TensorFlow default to **Nesterov** momentum (opt-in flag) for optimizers like SGD-with-momentum, but heavy-ball ($\beta \approx 0.9$) is also widely used.
- In both cases the *momentum* idea survives even where the proof doesn't. The inertia smooths noisy gradients and helps escape narrow ravines.

One-sentence rule

Smooth convex with a good L : Nesterov. Non-convex / tuned hand-knobs: heavy-ball. The *concept* of accumulating a velocity is what matters more than the exact update.

Stochastic gradient descent — when the gradient is expensive

The setup that makes deterministic GD infeasible

$$f(x) = \frac{1}{N} \sum_{i=1}^N f_i(x), \quad N \text{ in the millions.}$$

Computing $\nabla f = \frac{1}{N} \sum \nabla f_i$ touches every sample. SGD replaces this by a single random sample:

$$x_{k+1} = x_k - \alpha_k \nabla f_{i_k}(x_k), \quad i_k \sim \text{Unif}\{1, \dots, N\}.$$

Expected update matches GD: $\mathbb{E}[\nabla f_{i_k}] = \nabla f$. But each step costs N times less.

The price: variance

Stochastic gradients have variance $\sigma^2 = \text{Var}[\nabla f_i]$. That variance prevents convergence at a fixed step size — you oscillate inside a ball of radius $\sim \sqrt{\alpha \sigma^2 / \mu}$ around x^* .

Two remedies

- **Diminishing step sizes:** $\alpha_k = c/(k+1)$ gives asymptotic $O(1/k)$ rate on strongly convex problems.
- **Mini-batches:** average b samples per step. Variance drops by $1/b$; cost grows by b . Sweet spot typically $b \in [32, 512]$ — tune on validation loss.

SGD with momentum — the workhorse for neural nets

Even though the convergence theory is delicate, in practice SGD almost always runs *with* momentum. The update is structurally identical to heavy-ball, but with a stochastic gradient in place of $\nabla f(x)$:

```
# SGD with (heavy-ball) momentum
v = beta * v - alpha * stoch_grad(x, batch)
x = x + v
```

Why momentum survives the noise

Momentum v_k is a *moving average* of stochastic gradients — $v_k \approx -\alpha \sum_{j \leq k} \beta^{k-j} \hat{g}_j$. The variance of this average is lower than the variance of any single sample. You get de-noising for free.

Typical hyperparameter choices

- $\beta \in [0.9, 0.99]$ (this is momentum=0.9 in torch.optim.SGD).
- α : start aggressive, decay by cosine or step schedule.
- Nesterov variant: SGD(..., nesterov=True). Small gains typically; the headline effect is the inertia, not the lookahead.

Adam — momentum plus per-coordinate scaling

The idea in one sentence

Keep a momentum (m_t) and an exponential running variance (v_t). Step along m_t divided by $\sqrt{v_t}$ so that coordinates with consistently large gradients take smaller steps.

```
# Adam, per-coordinate
m = beta1*m + (1-beta1) * g
v = beta2*v + (1-beta2) * (g*g)
m_hat = m / (1 - beta1**t)      # bias correction
v_hat = v / (1 - beta2**t)
x = x - alpha * m_hat / (np.sqrt(v_hat) + eps)
```

What's actually happening

- m_t is momentum (Kingma–Ba call it the “first moment” but it’s the same thing we just saw).
- $\sqrt{v_t}$ is a diagonal approximation of the Hessian scaling. Adam is momentum + diagonal preconditioning — “momentum plus Jacobi.”

The trade-off that's come under scrutiny

Adam converges fast on noisy landscapes (transformers, RNNs), but the per-coordinate scaling can hurt generalization vs. well-tuned SGD with momentum on image tasks. The 2020s have seen a slight swing back toward plain SGD+momentum for supervised vision.

Convergence rates at a glance

Method	Smooth convex	L -smooth μ -strongly-convex
GD	$O(1/k)$	$(1 - 1/\kappa)^k$
Heavy-ball	(no general)	$(1 - 1/\sqrt{\kappa})^k$ on quadratics
Nesterov / FISTA	$O(1/k^2)$	$(1 - 1/\sqrt{\kappa})^k$
SGD (diminishing step)	$O(1/\sqrt{k})$	$O(1/k)$ (expected error)
SGD + momentum	same order as SGD	same order as SGD
Adam	heuristic	heuristic

“ κ ” is the condition number L/μ . The square root in the Nesterov row is the whole point of acceleration — optimal among methods that only see ∇f .

Why SGD is slower in rate but wins in practice

A single deterministic iteration costs $O(N)$; a stochastic one costs $O(1)$. SGD’s “slow” $O(1/k)$ buys you N -times-as-many iterations per unit wall clock. Crossover point is near the noise floor set by σ^2 .

Pedagogical takeaways — and a recipe

Three ideas, one common theme

Every method on the previous slide is a *variance-reduction trick on gradients*:

- **Nesterov**: reduce the variance introduced by curvature by evaluating gradients at lookahead points.
- **Momentum**: reduce the variance of *update directions* by accumulating across iterations.
- **SGD + mini-batch + momentum**: reduce the variance of *stochastic samples* by averaging in time.

A practical default recipe

1. **Deterministic, convex, smooth**: Nesterov / FISTA. Accept the L -smoothness assumption or add a backtracking line search.
2. **Deterministic, convex, non-smooth (prox-friendly)**: FISTA with proximal step — you already have this in NB 19.
3. **Stochastic, non-convex (deep learning)**: SGD with momentum ($\beta = 0.9$), cosine schedule; switch to Adam if training is unstable.
4. **When to stop tweaking**: once you've hit the variance floor for your step size, *decrease the step size* before changing the optimizer.

Roadmap

Part 0.5 — Python Toolchain: Packages, Environments, Conventions

Part I — Modeling Languages and Solver Interfaces

Part II — Implement It Yourself

Part III — Large-Scale Case Studies

Deep Dive — Acceleration, Momentum, and Stochastic Gradient

Summary

Summary: What the Scaling Experiments Tell Us

Headline numbers from the 23 notebooks

- **Sparse Hessians** (NB 1): $100\times$ speedup on 10k-var QP
- **CVXPYgen** (NB 3): $100\text{--}1000\times$ on repeated MPC solves
- **PDLP** (NB 7): solves a 10M-nonzero LP that IPM / simplex cannot
- **PCG + preconditioner** (NB 10): $100\times$ on 2-D Poisson vs. dense
- **L-BFGS storage** (NB 12): $500\times$ less memory, same iters
- **FISTA vs. ISTA** (NB 19): $O(1/k^2)$ vs. $O(1/k)$ visible
- **Mesh-independent PCG** (NB 22): **1** iter regardless of N
- **Heavy ball vs. GD** (NB 23): 48 vs. 692 iters on $\kappa = 100$

The case for “these tools are powerful” is not rhetorical. It is *empirical*, backed by log-log plots on real problems at real sizes.

Decision Tree: Which Tool for Which Problem?

You have...	Reach for...
Small, smooth, unconstrained	SciPy <code>minimize</code> with <code>method="trust-ncg"</code>
Convex, fits on paper	CVXPY (NB 2)
MPC / repeated parametric	CVXPYgen (NB 3)
Large NLP, convex or not	IPOPT via <code>pyomo</code> / <code>casadi</code> (NB 4)
MILP / MIQP	Gurobi; SCIP as open-source alternative
MINLP with polynomials / exp / log	SCIP spatial B&B (NB 6)
LP with millions of variables	PDLP / PDHG (NB 7)
Medium, equality-constrained, smooth	SQP / SLSQP (NB 8)
LASSO / ℓ_1 -regularized	FISTA (NB 19) or ADMM (NB 20)
Distributed / decomposable	ADMM (NB 20)
Sparse signal recovery	Basis pursuit / LASSO (NB 21)
Neural network training	Adam, then SGD+mom, then L-BFGS (NB 14)

Summary and Next Steps

Key Takeaways

- **Modeling languages** (CVXPY, Pyomo) separate *what* you want from *how* it's solved
- **Sparsity and structure** are *the* performance levers
- **Condition number** silently kills solutions — check it
- **First-order methods** scale to huge problems; second-order methods converge faster — choose based on the bottleneck
- Every algorithm you coded from scratch is inside a production solver

Next Time: course review & exam 2 logistics.

Questions?

Where the code lives

Repository

ISE 5406 / code / ise5406_notebooks /

Directory layout

- `part1_solvers/` — NBs 1–8, 15–17 (11 notebooks)
- `part2_from_scratch/` — NBs 9–12, 18–20, 22–23 (9 notebooks)
- `part3_case_studies/` — NBs 13, 14, 21 (3 notebooks)
- `common/ise5406_utils.py` — shared utilities
- `builders/build_NN_*.py` — regenerate any notebook
- `SOLUTIONS.md` — worked solutions to every coding exercise
- `html/` — pre-rendered HTML for offline viewing

To regenerate any notebook: `python builders/build_NN_*.py`.
Edit the Python source, not the `.ipynb` JSON.